

NBA Player Performance & Salary Analytics Pipeline

CPSC 482 – Data-Intensive Systems | Final Report
Jacob Jarvis
May 2026

1. Project Overview and Goals

This project implements a complete end-to-end data analytics pipeline on Google Cloud Platform that processes over 70 years of NBA player statistics and 30 years of salary data. The pipeline ingests raw CSV datasets, stores them in Cloud Storage, transforms them using PySpark on Dataproc, warehouses the cleaned data in BigQuery, trains machine learning models to predict player salaries, and presents the results through an interactive Looker Studio dashboard.

The primary goals were to demonstrate proficiency with GCP data services, to generate meaningful analytics from a large real-world dataset, and to build a salary prediction model that identifies overpaid and undervalued NBA players based on their on-court performance statistics.

2. Dataset Description

The project uses four interrelated CSV datasets sourced from Kaggle, spanning player box scores, season averages, individual salaries, and team payroll.

Dataset	Raw Rows	Size	Coverage	Clean Rows
Player Box Scores	1,309,778	158 MB	1950–2022	1,183,956
Player Season Stats	28,237	4.5 MB	1950–2022	25,732
Player Salaries	15,857	740 KB	1990–2023	15,203
Team Payroll	966	43 KB	1990–2023	966

The box score dataset is the largest at 158 MB and 1.3 million individual game records. It contains per-game statistics including points, rebounds, assists, shooting percentages, and plus/minus for every player who appeared in an NBA game from 1946 through 2022. The salary data enables a unique analytical angle: predicting what a player should be paid based on their statistics and identifying discrepancies.

Data quality considerations include missing values for statistics not tracked in earlier eras (three-point shooting before 1979, steals and blocks before 1973), currency formatting in salary fields, and duplicate entries for players who were traded mid-season.

3. Pipeline Workflow and Services Used

The pipeline follows a six-stage architecture using core GCP services:

- Cloud Storage (GCS):** Raw CSV files are uploaded to a GCS bucket (final-project-493902-nba-analytics) organized into raw/ and processed/ prefixes. GCS serves as the

landing zone for ingestion and the intermediate storage layer between processing and warehousing.

- **Dataproc (PySpark):** A single-node Dataproc cluster running PySpark handles the data transformation: reading CSVs from GCS, cleaning and standardizing schemas, computing derived features, and writing the output as Parquet files back to GCS.
- **BigQuery:** Processed Parquet files are loaded into BigQuery as native tables. Six analytics views are created on top of these tables to support dashboard queries and ML training.
- **BigQuery ML:** Two regression models (Linear Regression and XGBoost) are trained directly in BigQuery SQL to predict player salaries from performance statistics.
- **Looker Studio:** A five-page interactive dashboard connects to BigQuery views and tables, providing filterable visualizations of player performance, salary analysis, and model predictions.

Figure 1: GCS Bucket – Raw Data Files

Location	Storage class	Public access	Protection
us-central1 (Iowa)	Standard	Subject to object ACLs	Soft Delete

Objects	Configuration	Permissions	Protection	Lifecycle	Observability	Inventory Reports	Operations
Folder browser final-project-493902-nba-analytics processed/ raw/	Buckets > final-project-493902-nba-analytics > raw Create folder Upload Transfer data Create batch operations New Other services Learn						
Filter by name prefix only Filter objects and folders Show Live objects only							
Name	Size	Type	Created	Storage class	Last modified		
NBA Payroll(1990-2023).csv	44 KB	application/vnd.ms-excel	Apr 19, 2026, 7:51:50 PM	Standard	Apr 19, 2026, 7:51:50 PM		
NBA Player Box Score Stats(1950-2023).csv	158.4 MB	application/vnd.ms-excel	Apr 19, 2026, 7:52:23 PM	Standard	Apr 19, 2026, 7:52:23 PM		
NBA Player Stats(1950-2023).csv	4.7 MB	application/vnd.ms-excel	Apr 19, 2026, 7:51:52 PM	Standard	Apr 19, 2026, 7:51:52 PM		
NBA Salaries(1990-2023).csv	756.9 KB	application/vnd.ms-excel	Apr 19, 2026, 7:51:51 PM	Standard	Apr 19, 2026, 7:51:51 PM		

4. Data Processing Steps

Data processing was performed in two stages: first locally using pandas for development and testing, then on Dataproc using PySpark for the GCP pipeline demonstration.

4.1 Cleaning and Standardization

- **Column renaming:** All columns were renamed to consistent snake_case (e.g., PLAYER_NAME to player_name, FG_PCT to fg_pct).
- **Type casting:** Numeric fields were cast to appropriate types (INTEGER for counts, FLOAT for percentages). Date strings were parsed from formats like "NOV 19, 1946" to proper date types.
- **Currency cleaning:** Salary and payroll fields containing dollar signs and commas (e.g., "\$4,070,917") were stripped and cast to integers.
- **Filtering:** Box score rows with zero or null minutes were removed. Season stats rows with team = "TOT" (aggregate rows for multi-team players) were dropped.
- **Deduplication:** Records were deduplicated on composite keys (player_name + game_id for box scores, player_name + season + team for season stats).

4.2 Feature Engineering

- **Per-36-minute stats:** Points, rebounds, and assists were normalized to a per-36-minute basis to enable fair comparison across players with different minutes.
- **True Shooting Percentage:** Computed as $PTS / (2 * (FGA + 0.44 * FTA))$, providing a single efficiency metric that accounts for free throws and three-pointers.
- **Home/away flag:** Derived from the matchup string ("vs." indicates home, "@" indicates away).

4.3 Dataproc Execution

The PySpark job was submitted to a single-node Dataproc cluster (n1-standard-4, 100GB disk) and completed successfully, processing all four datasets and writing Parquet output to GCS.

Figure 2: Dataproc PySpark Job Output

```

C:\Users\back\Jarvis>nb-analytics-pipeline\nb-analytics-pipeline\gcp\csc\scripts\transform_pyspark.py --gs://final-project-493902-nba-analytics/scripts/
google recommends using Cloud Storage GCS (https://docs.cloud.google.com/storage/docs/discover-object-storage-gcloud) instead of gutil. Please refer to migration guide (https://docs.cloud.google.com/storage/
docs/migrating-file-urls-to-cloud-storage) to assist you.
C:\Users\back\Jarvis>nb-analytics-pipeline\nb-analytics-pipeline\gcp\csc\scripts\transform_pyspark.py [Content-Type=text/x-python]...
[1 files] 9.4 KiB / 9.4 KiB
Execution completed over 1 objects/9.4 MiB.
C:\Users\back\Jarvis>nb-analytics-pipeline\nb-analytics-pipeline\gcp\dataproc\jobs\submit_pyspark.py --final-project-493902-nba-analytics/scripts/transform_pyspark.py --cluster=nba-cluster --region=us-central1
Job ID:jarvis9856csc792919906c6f05f9 submitted.
Waiting for job output...
08/04/25 20:16:05 INFO SparkEnv: Registering MapOutputTracker
08/04/25 20:16:45 INFO SparkEnv: Registering BlockManagerMaster
08/04/25 20:16:45 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
08/04/25 20:16:45 INFO SparkEnv: Registering OutputCommitCoordinator
08/04/25 20:16:45 INFO DataProcSparkUIin: Registered 128 driver metrics
08/04/25 20:16:47 INFO DefaultHadoopMapReduceV2Provider: Connecting to ResourceManager at nbh-cluster-us-central-1-c-final-project-493902.internal/10.128.0.2:8032
08/04/25 20:16:47 INFO HadoopProxy: Connecting to Application History server at nbh-cluster-us-central-1-c-final-project-493902.internal/10.128.0.2:10200
08/04/25 20:16:47 INFO Configuration: resource-types.xml not found
08/04/25 20:16:47 INFO Configuration: Unable to find 'resource-types.xml'.
08/04/25 20:16:51 INFO VarnCClientImpl: Submitted application application_7777107622107_0801
08/04/25 20:16:52 INFO DefaultHadoopMapReduceV2Provider: Connecting to ResourceManager at nbh-cluster-us-central-1-c-final-project-493902.internal/10.128.0.2:8030
08/04/25 20:16:54 INFO GoogleCloudStorageImpl: Ignoring exception of type GoogleCSUnresponsiveException: verified object already exists with desired state.

NBA Data Transformation Pipeline

Loading box scores from gs://final-project-493902-nba-analytics/raw/NBA Player Box Score Stats(1980 - 2022).csv
Box score transformed: 112929 rows
Loading season stats from gs://final-project-493902-nba-analytics/raw/NBA Stats(1980 - 2022).csv
Season stats transformed: 20712 rows
Loading salaries from gs://final-project-493902-nba-analytics/raw/NBA Salaries(1990-2023).csv
Salaries transformed: 15283 rows
Loading payroll from gs://final-project-493902-nba-analytics/raw/NBA Payroll(1990-2023).csv
Payroll transformed: 960 rows

Writing processed data to GCS as Parquet...
08/04/25 20:17:07 WARN FileUtil: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
08/04/25 20:18:32 INFO GoogleCloudStorageFileSystem: Successfully repaired 'gs://final-project-493902-nba-analytics/processed/player_box_scores/' directory.
Writing player_season_stats to gs://final-project-493902-nba-analytics/processed/player_season_stats/
08/04/25 20:18:38 INFO GoogleCloudStorageFileSystem: Successfully repaired 'gs://final-project-493902-nba-analytics/processed/player_season_stats/' directory.
Loading player salaries to gs://final-project-493902-nba-analytics/processed/player_salaries/
08/04/25 20:18:41 INFO GoogleCloudStorageFileSystem: Successfully repaired 'gs://final-project-493902-nba-analytics/processed/player_salaries/' directory.
Loading team payroll to gs://final-project-493902-nba-analytics/processed/team_payroll/
08/04/25 20:18:43 INFO GoogleCloudStorageFileSystem: Successfully repaired 'gs://final-project-493902-nba-analytics/processed/team_payroll/' directory.

```

Figure 3: Dataproc Job Completion (status: DONE)

[illegible]

5. Results Obtained

5.1 BigQuery Tables

Four processed tables were loaded into BigQuery with the following verified row counts:

Figure 4: BigQuery Row Count Verification

1	player_season_stats	25732
2	player_salaries	15203
3	player_box_scores	1183956
4	team_payroll	966

Six analytics views were created to support the dashboard and ML workflows: v_player_season_averages, v_player_careers, v_league_trends, v_player_stats_with_salary, v_team_spending, and v_ml_salary_training.

Figure 5: BigQuery player_box_scores Schema (30 fields)

Field name	Type	Mode	Description	Key	Collation	Default Value	Policy Tags	Data Policies
season	INTEGER	NULLABLE						
game_id	INTEGER	NULLABLE						
player_name	STRING	NULLABLE						
team	STRING	NULLABLE						
game_date	TIMESTAMP	NULLABLE						
matchup	STRING	NULLABLE						
win_loss	STRING	NULLABLE						
minutes	INTEGER	NULLABLE						
fg_made	INTEGER	NULLABLE						
fg_attempted	INTEGER	NULLABLE						
fg_pct	REAL	NULLABLE						
fg3_made	INTEGER	NULLABLE						
fg3_attempted	INTEGER	NULLABLE						
fg3_pct	REAL	NULLABLE						
ft_made	INTEGER	NULLABLE						
ft_attempted	INTEGER	NULLABLE						
ft_pct	REAL	NULLABLE						
off_rebounds	INTEGER	NULLABLE						
def_rebounds	INTEGER	NULLABLE						
rebounds	INTEGER	NULLABLE						
assists	INTEGER	NULLABLE						
steals	INTEGER	NULLABLE						
blocks	INTEGER	NULLABLE						
turnovers	INTEGER	NULLABLE						
personal_fouls	INTEGER	NULLABLE						
points	INTEGER	NULLABLE						
plus_minus	INTEGER	NULLABLE						
pts_per36	REAL	NULLABLE						
reb_per36	REAL	NULLABLE						
ast_per36	REAL	NULLABLE						
true_shooting_pct	REAL	NULLABLE						
is_home	BOOLEAN	NULLABLE						

5.2 BigQuery ML Models

Two regression models were trained to predict a player's inflation-adjusted salary from their season statistics. The features used were: age, games played, minutes per game, points, rebounds, assists, steals, blocks, field goal percentage, three-point percentage, and effective field goal percentage.

Model	R ² Score	MAE	Explained Var.
Linear Regression	0.381	\$4.07M	0.383
XGBoost (Boosted Tree)	0.484	\$3.53M	0.487

The XGBoost model outperformed linear regression across all metrics, achieving an R² of 0.484 and reducing mean absolute error by approximately \$540K. The moderate R² reflects the reality that NBA salaries are influenced by many factors beyond on-court statistics, including draft position, marketability, team cap situations, and contract timing.

Figure 6: Model Evaluation Results

Row	model	mean_absolute_e...	mean_squared_er...	mean_squared_lo...	median_absolute...	r2_score	explained_variance
1	Linear Regression	4070917.895042...	32868053195171...	1.009736332149...	2900204.333855...	0.381316871268...	0.381329775115...
2	XGBoost	3531959.403941...	27419635213119...	0.682833789453...	2275157.25	0.483873729861...	0.486734239175...

5.3 Salary Predictions

The XGBoost model was used to generate predictions for all players in the training dataset. Players were then ranked by overpay amount (actual salary minus predicted salary) to identify the most overpaid players. The top overpaid players include Mike Conley (2020, \$31.3M overpay), Gordon Hayward (2019, \$28.7M overpay), and Paul George (2018, \$25.4M overpay) — cases where injuries or role changes reduced production while salaries remained locked in from earlier contract negotiations.

Figure 7: Most Overpaid Players (BigQuery Query Results)

Row	player_name	season	ppg	rpg	apg	age	actual_salary	predicted_salary	overpay_amount	overpay_pct
1	Mike Conley	2020	675.0	151.0	208.0	32.0	39656633	815704.0	31340929.0	376.9
2	Gordon Hayward	2019	825.0	322.0	244.0	28.0	37828768	9098955.0	28729813.0	315.7
3	Paul George	2018	1734.0	447.0	263.0	27.0	35935979	10573391.0	25362588.0	239.9
4	Klay Thompson	2019	1680.0	299.0	186.0	28.0	37876556	13404161.0	24472395.0	182.6
5	Blake Griffin	2021	260.0	121.0	63.0	31.0	35341704	11594240.0	23747464.0	204.8
6	Andrew Wiggins	2019	1321.0	352.0	184.0	23.0	31817869	9183909.0	22633960.0	246.5
7	Chandler Parsons	2019	187.0	70.0	43.0	30.0	29039054	6437780.5	22601274.0	351.1
8	Tobias Harris	2020	1411.0	495.0	227.0	27.0	39491945	17233890.0	22258055.0	129.2
9	Paul Millsap	2019	883.0	505.0	141.0	33.0	35109446	13029167.0	22080279.0	169.5
10	Blake Griffin	2021	245.0	104.0	77.0	31.0	35341704	13492221.0	21849483.0	161.9
11	Kyle Lowry	2019	926.0	312.0	564.0	32.0	38517776	16681556.0	21836220.0	130.9
12	Chris Paul	2019	906.0	265.0	473.0	33.0	44545016	22816210.0	21728806.0	95.2
13	Khris Middleton	2019	1407.0	461.0	331.0	27.0	35402639	13722166.0	21680473.0	158.0
14	Russell Westbrook	2019	1675.0	807.0	784.0	30.0	44545016	22888430.0	21655586.0	94.6
15	Chandler Parsons	2018	284.0	90.0	69.0	29.0	28347480	6735010.5	21612440.0	320.9
16	John Wall	2021	823.0	129.0	275.0	30.0	48332594	26755202.0	21570092.0	80.6
17	Chandler Parsons	2017	210.0	84.0	55.0	28.0	27957547	6420782.0	21536765.0	335.4
18	Jrue Holiday	2018	1537.0	365.0	486.0	27.0	31326979	9895293.0	21431686.0	216.6
19	Andrew Wiggins	2018	1452.0	358.0	160.0	22.0	29446649	8871262.0	21075387.0	237.6
20	John Wall	2019	663.0	116.0	279.0	28.0	44189315	23484912.0	20704403.0	88.2

6. Visualizations / Dashboard

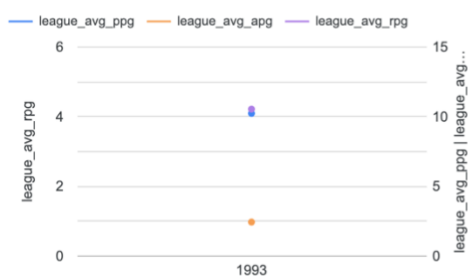
A five-page interactive dashboard was built in Looker Studio, connected directly to BigQuery views. Each page serves a distinct analytical purpose:

6.1 League Trends

Line charts showing league-wide scoring, rebounding, and assist averages from 1950 through 2022, alongside a dedicated three-point attempt trend chart that clearly visualizes the three-point revolution accelerating from the mid-2010s onward.

Figure 8: League Trends Dashboard Page

League Scoring, Rebounding & Assist Trends (...)



Three-Point Attempts Per Game By Season



6.2 Player Performance Explorer

A filterable table and bar chart allowing users to select any player and view their season-by-season statistics including PPG, RPG, APG, true shooting percentage, and minutes per game.

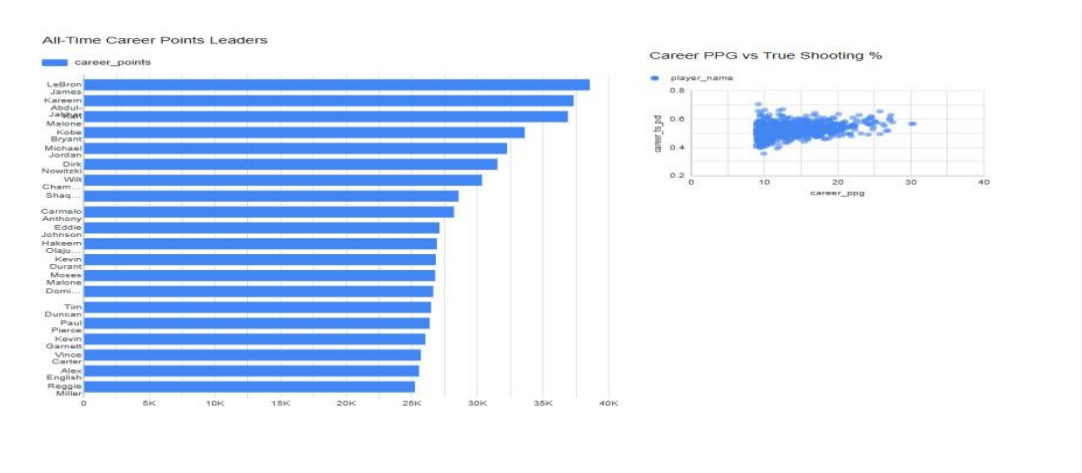
Figure 9: Player Performance Explorer



6.3 Career Leaders

Horizontal bar chart of all-time career points leaders and a scatter plot of career PPG versus true shooting percentage, revealing the relationship between volume scoring and efficiency.

Figure 10: Career Leaders Dashboard Page



6.4 Salary Analysis

A scatter plot of PPG versus salary showing the general positive relationship, alongside a table of the best-value players ranked by PPG per million dollars of salary.

Figure 11: Salary Analysis Dashboard Page



6.5 ML Predictions

Predicted versus actual salary scatter plot from the XGBoost model, plus a table of the most overpaid players with actual salary, predicted salary, and overpay percentage.

Figure 12: ML Predictions Dashboard Page



7. Challenges Encountered and Resolutions

- **GCP project ID vs. display name:** The project's display name ("final-project") differed from its ID ("final-project-493902"), causing repeated errors with gcloud, gsutil, and bq commands. Resolved by running gcloud projects list to discover the correct ID.
- **Billing configuration:** GCS bucket creation returned a 403 billing error because no billing account was linked. Resolved through the GCP console by linking a billing account to the project.
- **PowerShell vs. Bash:** The pipeline automation script was written for Bash, but development occurred on Windows PowerShell. Environment variable syntax (export vs \$env:), argument separators (--), and path conventions all required adaptation. Resolved by running commands individually with PowerShell-compatible syntax.
- **Dataproc argument passing:** PowerShell stripped the -- separator needed to pass arguments to PySpark scripts via gcloud dataproc jobs submit. Multiple workarounds failed. Resolved by hardcoding the bucket name in the PySpark script to eliminate the argument dependency entirely.

- **File naming:** CSV filenames contained spaces and parentheses (e.g., "NBA Player Box Score Stats(1950 - 2022).csv"), causing FileNotFoundError in Python scripts. Resolved by updating all file path references to match the exact filenames.
- **Missing historical statistics:** Several statistical categories (steals, blocks, turnovers, three-point stats) were not tracked by the NBA until the 1970s. These null values were retained rather than dropped, allowing basic scoring and rebounding analysis to span the full 1950–2022 range while advanced analysis is limited to modern eras.

8. Lessons Learned

- **Test locally before deploying:** Running the pandas transformation first caught schema and data quality issues without incurring Dataproc costs. The local Parquet output could also be uploaded directly to BigQuery as a fallback path.
- **Schema standardization is foundational:** Consistent column naming and type casting across sources was the most important preprocessing step. Without it, downstream joins and views would have failed silently or produced incorrect results.
- **Cost management requires deliberate design:** Using a single-node Dataproc cluster with auto-delete, BigQuery's free tier, and minimal GCS storage kept total project costs under \$3. This required conscious choices at each stage.
- **Iterative development prevents cascading errors:** Building the pipeline stage by stage with verification queries at each step (e.g., row count checks after loading) caught issues early before they propagated downstream.
- **ML models reveal domain complexity:** The moderate R^2 (0.48) of the salary prediction model is itself an insight: NBA salaries are only partially explained by on-court statistics. Contract timing, market dynamics, draft position, and team salary cap constraints all play significant roles that pure performance metrics cannot capture.

9. Potential Next Steps

- **Incorporate advanced metrics:** Adding PER, BPM, VORP, and Win Shares from Basketball Reference would strengthen the ML model's feature set and likely improve salary prediction accuracy.
- **Time-series forecasting:** Build models that predict next-season performance based on historical trajectory, age curves, and injury history.
- **Streaming ingestion:** Replace batch CSV ingestion with a Cloud Functions-based pipeline that pulls live game data from the NBA API on a nightly schedule, enabling near-real-time dashboard updates.
- **Team-level analysis:** Expand the payroll analysis to include win totals and playoff results, enabling a cost-per-win analysis at the team level.
- **Pipeline orchestration:** Add Cloud Composer (Airflow) to automate the full pipeline from ingestion through model retraining and dashboard refresh on a scheduled basis.